

Sparky

Hardware & Dashboard Demo Script

What's new since the last demo: real ESP32 hardware, WiFi onboarding, Sparky-branded admin

0. Recap - What's New Since Last Time

The previous demo (docs/demo_script.pdf) covered the software side only - no ESP32 existed yet, and the hardware wire protocol was shown simulated via a test page. Since then:

- Real ESP32-WROOM-32 hardware was sourced, flashed, and brought up.
- A branded WiFi captive portal (WiFiManager) now handles onboarding a Sparky unit onto a local network with no hardcoded credentials and no laptop/serial cable required after first flash.
- The admin dashboard was rebranded to Sparky, and a device delete feature (permanently remove a revoked device) was added alongside the existing register/revoke flow.
- The full software pipeline - audio in, transcription, tutor answer, synthesized answer out - was tested end-to-end against this real physical unit's real network stack and HTTP client, not just a browser or the protocol test page.

Scope, stated plainly so it isn't oversold: the mic/amp/speaker circuit hasn't been built yet - today's unit is bare ESP32 only. The firmware sends a pre-recorded test WAV (embedded in firmware) instead of a live recording, and there's nothing on the device yet to play the spoken answer back through. What this demo proves is that the real hardware's WiFi onboarding, HTTP client, and device authentication genuinely talk to the real backend end-to-end - the remaining gap is physical audio I/O, which is a wiring/assembly task, not a software one.

This script covers three demo segments in order: the dashboard, the captive portal (which also triggers the pipeline test automatically), and verifying that exchange server-side. CAD/enclosure work is a separate, ongoing track and isn't shown here.

1. Setup Checklist (do this before the audience arrives)

1. Start Ollama and confirm the default model is pulled (``ollama list``).
2. Start the backend: ``python simple_api.py`` from the repo root. Note the machine's LAN IP (not localhost/127.0.0.1) - the ESP32 needs a real address on the same WiFi network.
3. Open the admin dashboard in a browser at ``http://<your-lan-ip>:8001/admin`` and confirm it loads with current Sparky branding.
4. Power on the ESP32. If it should demo the captive portal fresh, trigger a WiFi-settings reset first (either the reset routine in the sketch, or hold the configured reset button/pin) so it boots straight into its own onboarding hotspot instead of reconnecting silently.
5. Have a phone or laptop ready to join the ESP32's onboarding hotspot on cue.

2. Sparky-Branded Admin Dashboard

ON SCREEN: `/admin`, `/admin/devices/view`, `/admin/interaction-log/view`

Quick tour of what changed here - this is the operator/teacher-facing side of the system.

- Sparky branding throughout - the dashboard, devices page, interaction log, document management,

diagnostics, and protocol test page all carry consistent Sparky styling now, not the generic RAG-system look from earlier.

Devices page:

- Register a device -> a one-time API key is shown, exactly once, to be copied into that unit's firmware (same hashed-key model as before - see the original demo script's device-auth Q&A if asked, it hasn't changed).
- Revoke a device (orange, reversible) vs. Delete a device (red, only available once already revoked, permanently removes it). Two-step by design - you can't accidentally hard-delete a live device.

Interaction log page - new device filter:

- Filter the log down to a single device's questions using the Device dropdown.
- With a device selected, an AI-generated activity summary appears: a short, plain-language narrative (built from the same local Ollama model, no cloud call) covering what topics that learner explored, signs of struggle, and what they seemed most engaged with - written for a teacher reviewing a learner's session, not a developer.

Good line for a project leader specifically: this turns a raw chat log into something a non-technical teacher could actually read and act on, entirely offline.

3. WiFi Captive Portal (this also fires the pipeline test)

ON SCREEN: on the physical ESP32 unit

This replaces hardcoded WiFi credentials in firmware - the real deployment story is a teacher or technician with no coding ability setting up a unit on-site. On this unit, the firmware is wired to automatically send its embedded test audio file the moment WiFi connects - so finishing the portal **is** the pipeline test, in one motion, not two separate demo steps.

1. Power on the ESP32 with WiFi settings cleared (see Setup step 4). It boots into its own access point instead of trying to connect to anything.
2. On a phone/laptop, join that hotspot. Most phones auto-open the portal page; otherwise navigate to the ESP32's AP address manually.
3. Show the portal page itself is Sparky-branded (custom head element/styling), not the generic WiFiManager default page.
4. Pick the real WiFi network from the scanned list, enter its password, submit.
5. The ESP32 reboots and joins the real network - and immediately, automatically, sends its embedded test audio file to the backend. Nothing else to press.

If asked why the portal matters: low-connectivity deployment sites won't always have someone who can flash firmware or edit a config file to change WiFi networks - this makes onboarding (and re-onboarding after a network change) a no-code, on-device process.

4. Verifying the Round-Trip, Server-Side

ON SCREEN: server console + admin Interaction Log + admin Devices

Because there's no mic or speaker on this unit yet, there is no audible confirmation to point to - verification happens entirely by checking the server independently agrees the exchange happened. Check all three; each is separate evidence, not just one screen repeated:

1. **Server console** (the terminal running `simple_api.py`) - the transcript and generated answer print live

as the request is processed. This is the most reliable evidence, since it comes straight from the backend, not from the device's own limited feedback.

2. **Admin Interaction Log**, filtered to this device - the new entry appears with the real transcript, answer, and grounding badge, tied to that specific device's ID.

3. **Admin Devices page** - that device's "last seen" timestamp updates to just now, independent confirmation the request really came from that physical unit's registered key.

Honest caveat if asked: this proves the ESP32's real WiFi stack, HTTP client, and device key genuinely reached the backend and got a correct answer back - not a simulation. It does not yet prove live microphone capture or audible playback, since that circuit isn't assembled. That's the next concrete milestone, not a hidden gap - worth saying so directly rather than implying more than what's built.

5. Other Things Worth Showing, If Time Allows

- **On-demand translation for QA review** (Text Chat / Interaction Log - Translate control): lets a reviewer check answer quality in a language they're more fluent in without changing the tutor's actual response language.

- **Response cache language fix:** a real bug where a cached answer could be served back in the wrong language after a language switch was found and fixed - good example of the kind of reliability issue that only shows up with multiple languages in real use, worth mentioning if the project leader asks about testing rigor.

- Everything in the original demo script (Sections 1-11 of docs/demo_script.pdf) still applies and hasn't changed - Text Chat, Voice Chat over the browser, document management, concurrency/load-test story, and multi-language support. This new script only covers what's new.

Anticipated Q&A - Hardware-Specific

Q: Why a captive portal instead of just hardcoding WiFi credentials in firmware?

A: Hardcoded credentials mean re-flashing firmware every time a unit moves networks. A captive portal lets anyone on-site - not just someone who can program the device - onboard or re-onboard it.

Q: What happens if the ESP32 loses WiFi mid-deployment?

A: It falls back to its own onboarding hotspot rather than failing silently, so it's always recoverable without physical reflashing - this fallback was specifically tested, not just assumed.

Q: How do you know the voice pipeline actually worked on real hardware, not just in a browser?

A: Confirmed independently via the backend's own server-side logs showing the real transcript and generated answer for that request, plus the resulting interaction log entry tied to that specific device's ID - not just trusting the device's own limited on-screen feedback.

Q: Can a device be permanently removed if it's lost, stolen, or decommissioned?

A: Yes - revoke immediately disables its key; delete (only available after revoking) permanently removes the device record. Two-step so a live device can't be hard-deleted by accident.

Q: What's the AI-generated activity summary actually useful for?

A: Gives a teacher a plain-language read on one learner's session - topics covered, signs of struggle, engagement - without them having to read a raw transcript, and without any cloud call.

Q: Why isn't the unit actually talking today - no live mic, no spoken answer?

A: The mic/amp/speaker circuit hasn't been assembled yet - this unit is bare ESP32 for now, running a firmware build that sends an embedded test recording instead of a live one. Today's demo proves the harder, less visible part - real WiFi onboarding, HTTP, and device auth all working against actual hardware. Wiring up audio I/O is a follow-on assembly task, not a software unknown.